

ReDISchool

Week 8

Loops

The same thing over, and over, and over, and over...



Agenda

01 Homework Recap

02 Arrays Recap

03 For Loops

04 For-in Loops

05 While Loops

Week 8

Homework Recap



Homework Discussion - 10 min

Week 8

Arrays Recap (optional)



Recap

Array Iterator Methods: reduce()




```
56
57    const totalNumberOfLetters = students.reduce(
58        (accumulator, student) => {
59            accumulator = accumulator + student.length
60            return accumulator
61    }, 0)
62
```

- We initialize the accumulator with 0.
- We apply the reduce method on each of the student names. We retrieve the name length and add it to the accumulator.
- Finally, we return the accumulator.



Recap

How to get elements from an array



You can access and retrieve elements from an array using its index.
You need to use the square bracket syntax to access array elements.

```
const element = array[index];
```

When you're accessing elements using index it looks like this:

```
const salad = ['🍎', '🍄', '🥦', '🥬', '🫑', '🥕', '🥑'];
salad[0]; // '🍎'
salad[2]; // '🥦'
salad[5]; // '🥕'
```

```
const info = ['Katherine', 'Johnson', '101'];
const firstName = info[0]
const lastName = info[1]
const age = info[2]
```



Recap

Array.isArray()

```
Array.isArray([27, false, 'ReDI School', ,(a,b)=>a+b, {}]); // returns true
Array.isArray('🍎'); // returns false
Array.isArray({ 'tomato': '🍎'}); // returns false
Array.isArray([]); // returns true
```



Recap

push():

```
const countries = ['Syria', 'Iran', 'Afghanistan', 'Lebanon']
countries.push('Yemen')
console.log(countries) // ['Syria', 'Iran', 'Afghanistan', 'Lebanon', 'Yemen']
```

```
const countries = ['Syria', 'Iran', 'Afghanistan', 'Lebanon']
countries.unshift('Yemen')
console.log(countries) // ['Yemen', Syria', 'Iran', 'Afghanistan', 'Lebanon']
```

Recap

pop():

```
const countries = ['Syria', 'Iran', 'Afghanistan', 'Lebanon']
const removedElement = countries.pop() // 'Lebanon'
console.log(countries) // [Syria', 'Iran', 'Afghanistan']
```

```
const countries = ['Syria', 'Iran', 'Afghanistan', 'Lebanon']
const removedElement = countries.shift() // 'Syria'
console.log(countries) // ['Iran', 'Afghanistan', 'Lebanon]
```

Recap

Array Iterator Methods: filter()



The filter() method creates a new array with all the elements that satisfies the condition mentioned in the function. Let's find the students whose name includes an **r**.

```
44
45    const studentsWithR = students.filter((element, index) => {
46        return element.includes('r')
47    })
48
```

```
34
35    let students = ['Alex', 'Javi', 'Maria', 'Jack', 'Ibra', 'Rhona']
36
```



Recap

Array Iterator Methods: map()



The map() method creates a new array by iterating through the elements and applying logic we provided in the function as an argument. We will make all of the students names be in capital letters.

```
51
52    const studentsUppercase = students.map((element, index) => {
53        return element.toUpperCase()
54    })
55
```



Recap

Array Iterator Methods: reduce()




```
56
57    const totalNumberOfLetters = students.reduce(
58        (accumulator, student) => {
59            accumulator = accumulator + student.length
60            return accumulator
61    }, 0)
62
```

- We initialize the accumulator with 0.
- We apply the reduce method on each of the student names. We retrieve the name length and add it to the accumulator.
- Finally, we return the accumulator.



Week 8

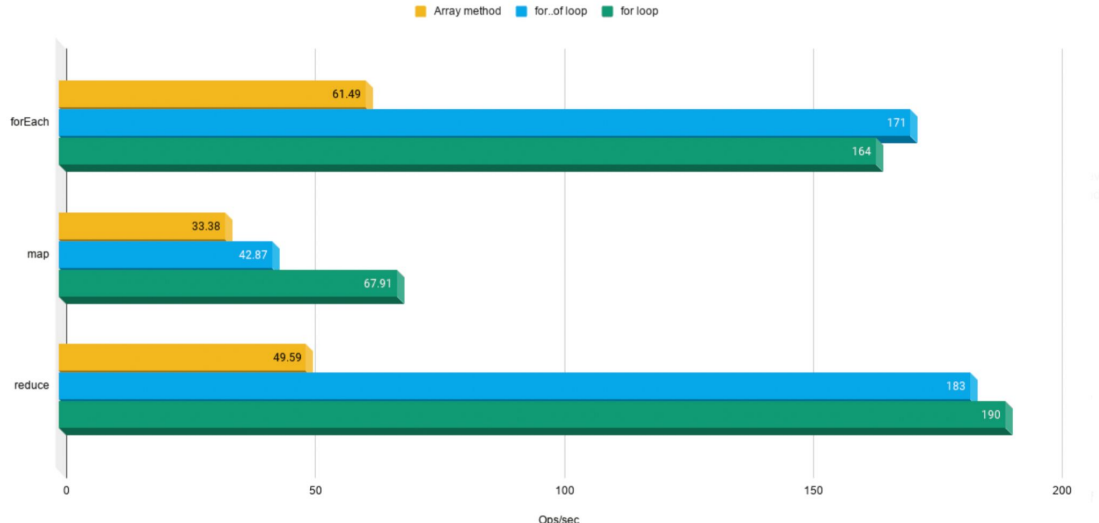
Loops



Loops in JavaScript

Loops are used in JavaScript to perform repeated tasks based on a condition. A loop will continue running until the defined condition returns false.

We have already worked with loops when we applied the array iterator methods the previous class. Array methods are easy to use, they are beautiful to look at, to read, to write, and declarative. But ***when performance actually matters***, other approaches like for-loops are multiple times faster.



Loops, visualized



For loop

Loops are handy, if you want to run the same code over and over again, each time with a different value.

```
for (initialization; condition; finalExpression) {
  // the code to run
}
```

initialization - This expression runs before the execution of the first loop, and is usually used to create a counter.

condition - This expression is checked each time before the loop runs. If it evaluates to true, the statement or code in the loop is executed. If it evaluates to false, the loop stops. And if this expression is omitted, it automatically evaluates to true.

block - This is where the code lives and it runs after the condition is checked

finalExpression - This expression is executed after each iteration of the loop. This is usually used to increment a counter, but can be used to decrement a counter instead.



For loop

```
for (let i = 0; i <= 3; i++) {
  console.log(i+i);
}
```

2nd loop
i = 1
log = 2

3rd loop
i = 2
log = 4

4th loop
i = 3
log = 6

5th loop
i = 4



For loop over an array

When iterating over an array, it's easy to accidentally exceed the bounds of the array.

For example, your loop may try to reference the 4th element of an array with only 3 elements:

```
const arr = [ 1, 2, 3 ];

for (let i = 0; i <= arr.length; i++) {
  console.log(arr[i]);
}
```

```
1                                                    index.js:138
2                                                    index.js:138
3                                                    index.js:138
undefined                                            index.js:138
```



For loop over an array

There are two ways to fix this code: set condition to either i < arr.length or i <= arr.length - 1.

```
for (let i = 0; i < arr.length; i++) {
  console.log(arr[i]);
}
```

```
for (let i = 0; i <= arr.length - 1; i++) {
  console.log(arr[i]);
}
```



Exercise

< 10 minutes >

1. Create an array containing 3 different online shops' names

2. Loop through that array using a for loop and console.log the name of the shop

```
for (initialization; condition; finalExpression) {
  // the code to run
}
```



Break

10 min

For-of loop

The JavaScript **for-of** statement loops through the values of an iterable object.

It lets you loop over iterable data structures such as Arrays, Strings, Maps, NodeLists, and more:

```
const cars = ['BMW', 'Volvo', 'Mini'];

for (let x of cars) {
  console.log(x)
}
```



For-of loop

The JavaScript **for-of** statement loops through the values of an iterable object.

It lets you loop over iterable data structures such as Arrays, Strings, Maps, NodeLists, and more:

```
const listContainer = document.getElementById('List-container');
const carBrands = ['BMW', 'Volvo', 'Saab', 'Ford', 'Fiat', 'Audi'];

for (let x of carBrands) {
  const node = document.createElement('li');
  node.innerHTML = x;
  listContainer.appendChild(node);
}
```



Exercise

< 10 minutes >

1. Create a <ul> element in your HTML
2. Loop through your database array using a for-of loop and do the following on each iteration:
    a. Create a <li> element
    b. Add the shop name to the innerHtml of the element
    c. Append the element to the <ul> created above



Break & Continue

The break statement "jumps out" of a loop.

The continue statement "jumps over" one iteration in the loop.

```
for (let i = 1; i < 10; i += 2) {
  if (i === 7) {
    break;
  }
  console.log('Total elephants: ' + i);
}
```

```
for (let i = 1; i < 10; i += 2) {
  if (i === 7) {
    continue;
  }
  console.log('Total elephants: ' + i);
}
```



Break & Continue

The break statement "jumps out" of a loop.

The continue statement "jumps over" one iteration in the loop.

```
for (let i = 1; i < 10; i += 2) {
  if (i === 7) {
    break;
  }
  console.log('Total elephants: ' + i);
}
// Output:
// Total elephants: 1
// Total elephants: 3
// Total elephants: 5
```

```
for (let i = 1; i < 10; i += 2) {
  if (i === 7) {
    continue;
  }
  console.log('Total elephants: ' + i);
}
```



Break & Continue

The break statement "jumps out" of a loop.

The continue statement "jumps over" one iteration in the loop.

```
for (let i = 1; i < 10; i += 2) {
  if (i === 7) {
    break;
  }
  console.log('Total elephants: ' + i);
}
// Output:
// Total elephants: 1
// Total elephants: 3
// Total elephants: 5
```

```
for (let i = 1; i < 10; i += 2) {
  if (i === 7) {
    continue;
  }
  console.log('Total elephants: ' + i);
}
// Output:
// Total elephants: 1
// Total elephants: 3
// Total elephants: 5
// Total elephants: 9
```



While loops

Loops can execute a block of code as long as a specified condition is true.

The while loop loops through a block of code as long as a specified condition is true.

```
let i = 1;
while (i < 4) {
  console.log(i);
  i++;
}
```



While loops

Loops can execute a block of code as long as a specified condition is true.

The while loop loops through a block of code as long as a specified condition is true.

```
let i = 1;
while (i < 4) {
  console.log(i);
  i++;
}
```

```
// Output:
// 1
// 2
// 3
```



Do-While loops

The do while loop is a variant of the while loop. The loop will always be executed at least once, even if the condition is false, because the code block is executed before the condition is tested:

```
let i = 3;
do {
  console.log('the number is ' + i);
  i++;
} while (i < 7);
```

```
// Output:
// 3
// 4
// 5
// 6
```



Exercise

< 10 minutes >

Play around with loops in the generative art example provided in Slack



Homework

https://github.com/ReDISchoolDK/Spring25_Frontend/blob/main/Week-08_Loops/homework/README.md

That’s it!

